

Dynamic Reach Extension in Compliant Robots: A Machine Learning Approach

Coppola Riccardo (10773170), Parma Valentino (10780461), Pezzi Francesco (11093867)

Abstract

This report investigates the exploitation of structural compliance and passive dynamics to extend the operational reach of an impedance-controlled robot mounted on a flexible base. Modeled as a two-degree-of-freedom mass-spring-damper system, the objective is to optimize a mechanical excitation signal (chirp) that leverages resonance to achieve target positions strictly beyond the robot’s nominal geometric limits. The problem is formulated as a Machine Learning optimization problem, comparing three distinct physics-informed pipelines: Direct Bayesian Optimization (BO), Surrogate-Assisted BO utilizing a Neural Network (NN) to accelerate computations, and a Genetic Algorithm (GA) paired with a non-dimensionalized NN trained to predict the displacement across varying robot configurations.

1 Introduction

In modern mechatronics and robotics, systems are often designed to be highly rigid to maximize precision and predictability. However, exploiting structural compliance can unlock extended dynamic capabilities, reduce energy consumption, and allow robots to perform tasks beyond their nominal geometric limits. This project explores this paradigm by tackling a dynamic control challenge: extending the reach of an impedance-controlled robot mounted on a compliant base by intentionally exciting the system’s passive dynamics.

The physical system is modeled as a two-mass-spring-damper system connected in series. The actuated degree of freedom (DoF) represents the robot’s end-effector, whose position is strictly bounded by operative hardware limits. This mass is coupled to a passive DoF representing a compliant base, which cannot be controlled directly. The primary objective is to command the actuated mass in such a way that it injects energy into the passive base, exploiting mechanical resonance to achieve a target position that lies strictly beyond the nominal reaching limits of the robot.

This project frames the mechanical challenge as a Machine Learning (ML) optimization and control problem. The core intent of this work is not just to solve the reachability task, but to establish clear, physics-informed ML pipelines, defining rigorous inputs, outputs, and architectures tailored to this mechatronic system. To this end, we propose, implement, and compare two distinct methodologies.

1. Traditional and Surrogate Bayesian Optimization: An open-loop parametric approach that optimizes a predefined excitation signal to safely maximize the robot’s reach. The Bayesian search is performed both on the original physical system and on a Neural Network (NN) trained as a more efficient surrogate of the physical simulator.

riccardo.coppola@mail.polimi.it, valentino.parma@mail.polimi.it, francesco2.pezzi@mail.polimi.it

2. Optimization search with genetic algorithm (GA) : A NN trained to approximate the general dynamic of spring mass damper systems with dimensionless parameters, is used to solve a grid search through GA and LHS distribution.

By developing both approaches, the project provides a comparative analysis of their frameworks, evaluating their respective sample efficiencies, architectures, and robustness in solving the case study.

2 Physical Environment Dynamics

To effectively train and evaluate the ML models, it is necessary to first formalize the physical environment. The system is modeled as a 1D, two-mass-spring-damper mechanism connected in series. This mathematical representation encapsulates the core dynamics of a impedance-controlled robot operating on a compliant, unactuated base.

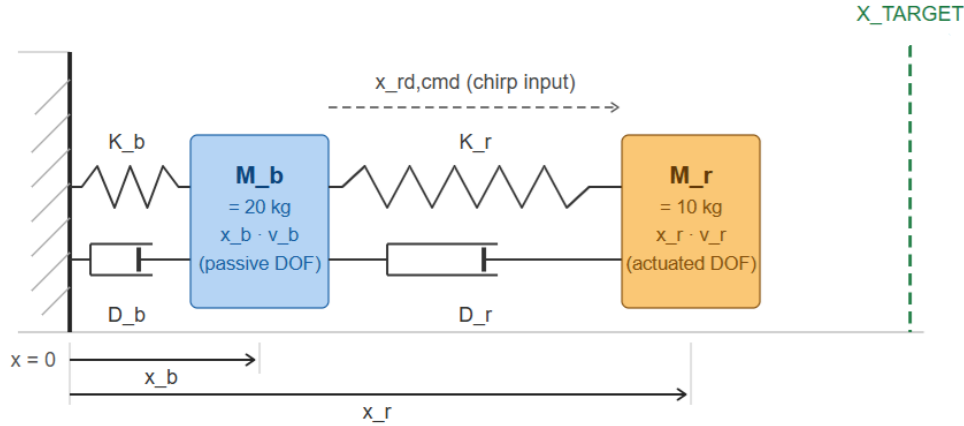


Figure 1: Physical System Scheme

2.1 System Architecture and Kinematics

The system consists of two distinct masses:

- **The Compliant Base (M_b):** This represents the unactuated degree of freedom. Its absolute position relative to a fixed world frame (the wall) is denoted as x_b .
- **The Robot End-Effector (M_r):** This represents the actuated degree of freedom. Its absolute position relative to the world frame is denoted as x_r .

To enhance the realism of the dynamic model, the finite widths of both the base (w_b) and the robot (w_r) are explicitly incorporated. By evaluating forces at the physical boundaries of the bodies rather than their centers of mass, the model accurately accounts for their geometry. Hence, the actual physical gap (x_{gap}) between the right face of the base and the left face of the robot is defined as:

$$x_{gap} = \left(x_r - \frac{w_r}{2} \right) - \left(x_b + \frac{w_b}{2} \right)$$

2.2 Passive Base Dynamics

The base mass M_b is anchored to a fixed wall ($x = 0$), coupled by a linear spring with stiffness K_b and a viscous damper with coefficient D_b . The force exerted by this ground connection on the base is given by:

$$F_b = K_b x_b + D_b \dot{x}_b$$

Because the base is strictly passive, its motion is driven entirely by the reaction forces generated by the robot’s movement overcoming this grounding force.

2.3 Impedance Control Formulation

A crucial aspect is the formulation of the robot actuation. The robot is not modeled using rigid position control, but it is governed by a **Cartesian Impedance Control**. In this context, the motor acts as a virtual spring-damper system connecting the actuator setpoint to the physical robot mass M_r . The impedance parameters are defined by a virtual stiffness K_r and virtual damping D_r . The control algorithm commands a desired relative displacement, $x_{rd,cmd}$, which in this case will be a target chirp signal. This command dictates the desired physical gap between the two masses, offset by the initial equilibrium resting length of the spring (x_{spring}):

$$x_{gap,desired} = x_{spring} + x_{rd,cmd}$$

The interaction force (F_{int}) pulling the robot mass toward the desired setpoint is proportional to the error between the actual gap and the commanded gap, alongside the relative velocity error:

$$F_{int} = K_r \left(x_{gap} - x_{gap,desired} \right) + D_r \left(\dot{x}_{gap} - \dot{x}_{rd,cmd} \right)$$

This force-based formulation is vital for the ML application, as it natively simulates the mechanical lag, overshoot, and resonance that occurs when commanding a system with physical inertia.

2.4 Equations of Motion

The coupled differential equations governing the system dynamics are obtained by applying Newton’s Second Law. The base mass M_b is acted upon by the ground constraint pulling it backward, and the interaction force from the robot pulling it forward:

$$M_b \ddot{x}_b = F_{int} - F_b$$

The robot mass M_r is driven solely by the impedance interaction force:

$$M_r \ddot{x}_r = -F_{int}$$

These equations are integrated numerically over time using a semi-implicit Euler scheme within the custom Python environment.

2.5 Mechanical Safety Constraints

To prevent the optimization algorithms from exploiting the simulation by commanding infinite control signals, strict bounds are imposed on the robot’s actuation. Specifically, the commanded relative position $x_{rd,cmd}$ is mathematically clipped to a maximum stroke amplitude limit ($A_{max} = 0.25$ m). Furthermore, a physical mechanical hard stop is embedded in the environment. If the actual relative gap x_{gap} exceeds a predefined structural limit (max_gap_r), the relative velocity of the robot is clamped. This simulates the absolute physical limits of the linear rails, forcing the ML agents to learn how to inject energy smoothly via resonance rather than through destructive brute force.

2.6 Excitation Signal: Exponential Chirp Command

To exploit the natural compliance of the system, the reference trajectory $x_{rd,cmd}$ fed to the impedance controller is defined as an exponential chirp signal. A chirp signal is uniquely suited for this mechatronic challenge because it continuously sweeps through a broad spectrum of frequencies over a specified duration,

allowing the system to naturally transition through its inherent resonant modes. The position command is formulated as a sinusoidal wave with a defined amplitude A (where $A \leq A_{max}$) and an exponentially increasing instantaneous frequency. The signal is governed by an initial starting frequency f_0 , a frequency multiplier k and the total excitation duration T . The mathematical formulation of the commanded relative position $x_{rd,cmd}(t)$ and its corresponding velocity $\dot{x}_{rd,cmd}(t)$ are given by:

$$x_{rd,cmd}(t) = A \sin(\Phi(t)) \quad \dot{x}_{rd,cmd}(t) = A \dot{\Phi}(t) \cos(\Phi(t))$$

where the instantaneous phase $\Phi(t)$ and angular velocity $\dot{\Phi}(t)$ are defined as:

$$\Phi(t) = 2\pi f_0 T \frac{k^{t/T} - 1}{\ln(k)} \quad \dot{\Phi}(t) = 2\pi f_0 k^{t/T}$$

The core objective for the ML frameworks is to discover the optimal set of chirp parameters, specifically f_0 , k , T , and A , that injects the correct amount of energy into the passive base. Crucially, the absolute target position (X_{target}) is deliberately chosen to lie strictly beyond the robot’s nominal kinematic reach, that is, $X_{target} > x_{r,0} + \frac{w_r}{2} + A_{max}$, where $x_{r,0}$ is the initial absolute resting position of the robot’s center and w_r is the robot’s width. Hence, the task cannot be achieved simply by extending the actuator to its limit. The agents must induce mechanical resonance to fling the robot end-effector to X_{target} at the peak of its dynamic oscillation, all while respecting the mechanical hard stop boundaries.

3 Bayesian Optimization

To achieve the target extension X_{target} , the system must be excited at a specific resonant frequency with a precise amplitude and duration. Deriving an analytical solution for the optimal chirp parameters is mathematically intractable, therefore, this approach treats the system as a "black box" and employs Bayesian Optimization (BO). BO is highly sample-efficient, making it ideal for optimizing expensive black-box functions. It builds a probabilistic model (a Gaussian Process) of the objective function and uses an acquisition function to balance exploration of uncertain parameter regions and exploitation of known optimal regions. We developed two distinct BO pipelines to solve this problem and evaluate computational efficiency.

3.1 Approach A: Direct BO

In the standard approach, the Bayesian Optimizer interacts directly with the numerical physics simulator.

1. Objective Function and Parameter Space: The search space is defined by four continuous parameters governing the exponential chirp signal: (f_0, k, T, A) . For each set of parameters proposed by the optimizer, the Python environment runs a full time-step integration of the system dynamics. The environment records the maximum absolute position reached by the right face of the robot, x_{max} . The objective function to be maximized by the BO algorithm is the negative absolute error between the maximum reach and the target:

$$\text{Objective}(f_0, k, T, A) = -|x_{max} - X_{target}|$$

2. Monte Carlo Robustness Analysis: While an optimizer can identify a set of parameters that perfectly achieves the target in a deterministic simulation, real-world hardware is inherently subject to wear, manufacturing tolerances, and unmodeled friction. Furthermore, all physical parameters are affected by measurement uncertainties, which complicates the modeling process. Notably, the damping coefficient is rarely known with precision and is typically estimated, introducing additional discrepancies between the simulated behavior and the actual hardware performance. To validate the robustness of the optimal chirp found by the BO, a Monte Carlo domain randomization analysis is performed. The optimal chirp parameters (f_0^*, k^*, T^*, A^*) are "frozen," and the simulation is run repeatedly for $N = 100$ iterations.

In each iteration, the physical parameters of the plant, specifically M_r, M_b, K_r, K_b , are perturbed by a Gaussian noise with a $\pm 10\%$ physical variance. The resulting distribution of x_{max} reveals whether the found resonant frequency is highly sensitive to exact mass or robust enough for real-world deployment.

3. Nominal BO Results: Applying this Direct BO framework, the optimizer successfully identified a resonant trajectory, with results detailed in Table 1: The optimized chirp parameters achieve the target extension of 0.75 m with a sub-millimeter precision error of just -0.66 mm.

Parameter / Metric	Symbol	Value
<i>Optimal Control Parameters</i>		
Initial Frequency	f_0	0.4182 Hz
Frequency Multiplier	k	4.9960
Excitation Duration	T	34.35 s
Chirp Amplitude	A	0.1770 m
<i>Simulation Performance</i>		
Target Position	X_{target}	0.750000 m
Maximum Reach Achieved	x_{max}	0.750486 m
Positioning Error	e	+0.000486 m

Table 1: Optimal Chirp Parameters and Nominal BO Performance

4. Monte Carlo Robustness Results: Following the nominal optimization, the robustness of the discovered chirp parameters was evaluated through a Monte Carlo domain randomization framework. The statistical results are summarized in Table 2 and Figure 2.

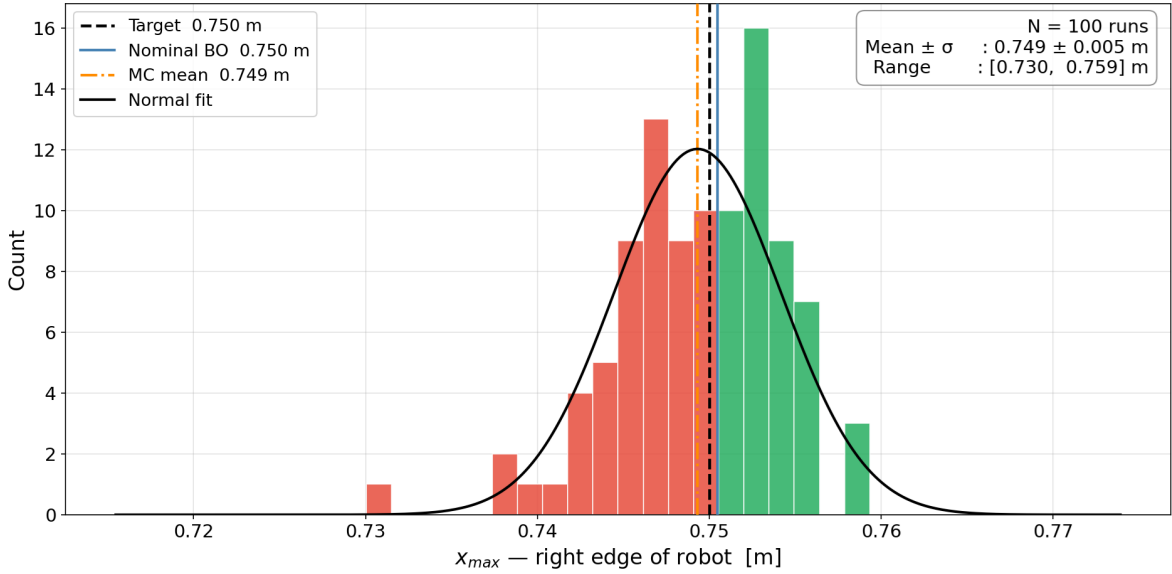


Figure 2: Monte Carlo Results Distribution

The convergence of the mean and standard deviation after 100 Monte Carlo iterations is shown in Figure 3.

Statistical Metric	Symbol	Value (m)
Empirical Mean	μ	0.7493
Standard Deviation	σ	0.0049

Table 2: Monte Carlo Robustness Results

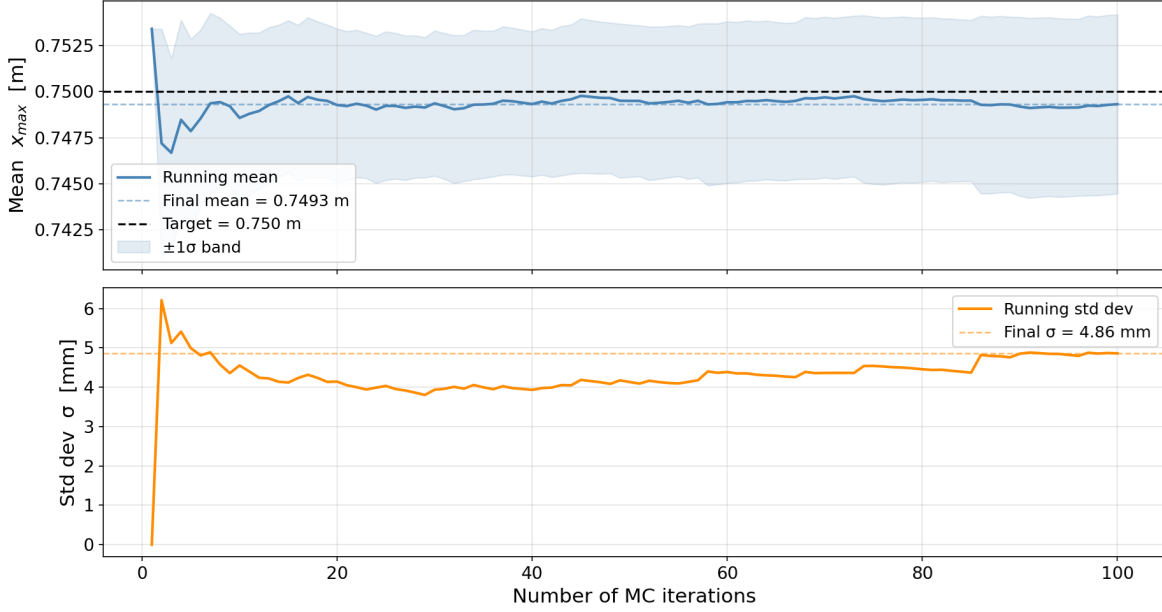


Figure 3: MC Mean and Std Convergence

The control strategy is resilient to modeling inaccuracies. The empirical mean remains exceptionally close to the 0.75 m target, and the standard deviation remains pretty narrow (4.9 mm). This confirms that the excitation parameters induce a broad, stable resonant band rather than a fragile peak.

3.2 Approach B: Surrogate-Assisted Bayesian Optimization

Running a full numerical ODE solver for every proposed parameter set becomes computationally prohibitive during large-scale hyperparameter tuning or when adapting to changing targets in real-time. To improve the numerical efficiency, we developed a Surrogate-Assisted pipeline using Deep Learning.

1. Dataset Generation and Surrogate Architecture: An automated script uniformly samples 5000 random parameter combinations (f_0, k, T, A) within the operational bounds. Each combination is evaluated on the numerical simulator to extract the resulting x_{max} . This creates a labeled dataset mapping the control inputs to the physical outcome.

2. Neural Network Surrogate Training: To learn the non-linear mapping, a Multi-Layer Perceptron is constructed using PyTorch. The network architecture consists of an input layer accepting the four normalized control parameters, followed by three hidden layers containing 128, 128, and 64 neurons, respectively. Each hidden layer utilizes a ReLU activation function, ending in a single-neuron linear output layer that predicts x_{max} . The NN is trained on the generated dataset using a Mean Squared Error loss function, and, once trained, it acts as a "digital twin" of the mechatronic system, capable of predicting the outcome of the oscillation in a fraction of a millisecond.

3. Surrogate Optimization and Validation: In this pipeline, the Bayesian Optimizer runs directly on the trained neural network, and the objective function queries the Neural Network's forward pass:

$$\text{Objective}_{\text{surrogate}}(f_0, k, T, A) = -|\text{NN}(f_0, k, T, A) - X_{\text{target}}|$$

Because the NN inference is orders of magnitude faster than numerical integration, the optimization campaign is expected to be much faster than in the case of Direct BO.

4. Performance and Computational Speedup: To ensure the integrity of the approach, the final "optimal" parameters proposed by the surrogate-BO are extracted and executed once on the true physics simulator, to analyze the discrepancy between the NN's predicted x_{max} and the simulator's true x_{max} . To quantify the benefits of the surrogate model, a direct performance comparison was conducted

between the Simulator-in-the-Loop BO and the NN-in-the-Loop BO for the same target ($X_{target} = 0.75$ m), utilizing identical optimizer settings. The results are summarized in Table 3.

Metric	Direct BO (Simulator)	Surrogate BO (NN)
Optimization Time	24.91 s	11.96 s
Predicted x_{max}	N/A	0.751624 m
True Validated x_{max}	0.750486 m	0.750602 m
Absolute Target Error	0.486 mm	0.602 mm
<i>Found Parameters</i>		
f_0 (Hz)	0.4182	1.6731
k	4.9960	1.9631
T (s)	34.35	29.01
A (m)	0.1770	0.1766

Table 3: Performance comparison: Direct BO vs. Surrogate-Assisted BO

The data highlights a fundamental engineering trade-off. The Surrogate BO achieved a true validated achieves a target error of roughly 0.6 mm. While this is slightly worse than the 0.48 mm error achieved by the Direct BO, the Surrogate BO completed the search in less than half the time (11.96 s vs 24.91 s). This demonstrates that while the NN introduces a minor approximation error into the objective landscape, it provides a substantial reduction in computational cost, making it highly advantageous for scenarios requiring rapid, iterative control generation.

5. Monte Carlo Robustness Results: Similarly as for the Direct BO, the solution of the Surrogate-Assisted BO was evaluated through a Monte Carlo randomization framework. The statistical results are summarized in Table 4 and Figure 2.

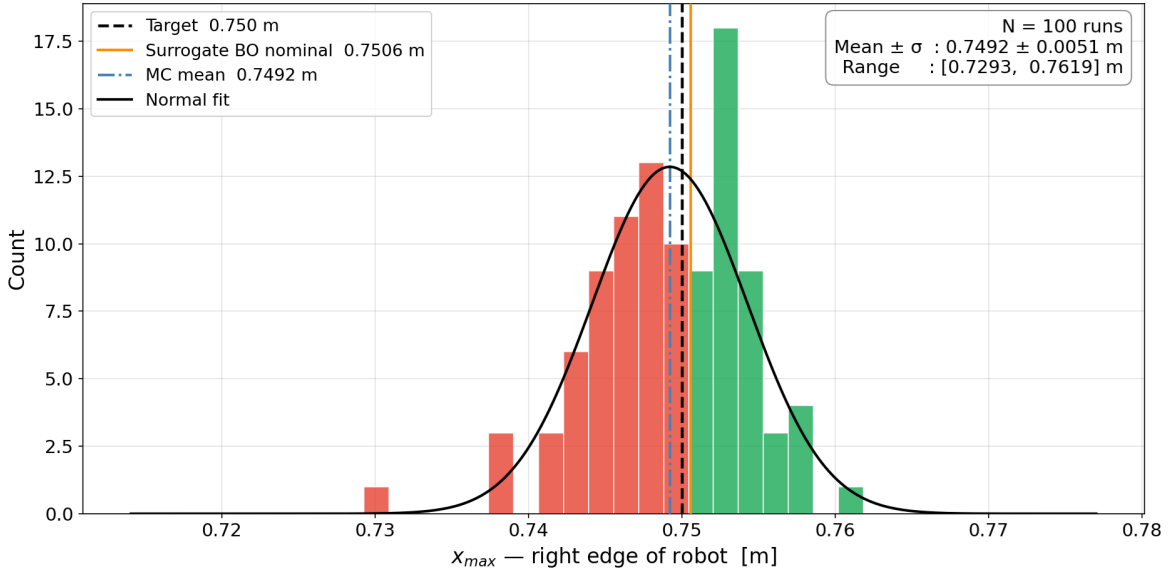


Figure 4: Monte Carlo Results Distribution

The convergence of the mean and standard deviation after 100 Monte Carlo iterations is shown in Figure 5. As observed in Figure 5, the statistical metrics successfully stabilize within the 100 Monte Carlo iterations, confirming the reliability of the sample size. Furthermore, the overall distribution satisfactory. Despite exhibiting a slightly larger absolute positioning error compared to the Direct BO pipeline, the results remain closely bounded around the empirical mean ($\sigma = 0.0051$ m). This confirms that the surrogate-

Statistical Metric	Symbol	Value (m)
Empirical Mean	μ	0.7492
Standard Deviation	σ	0.0051

Table 4: Monte Carlo Robustness Results

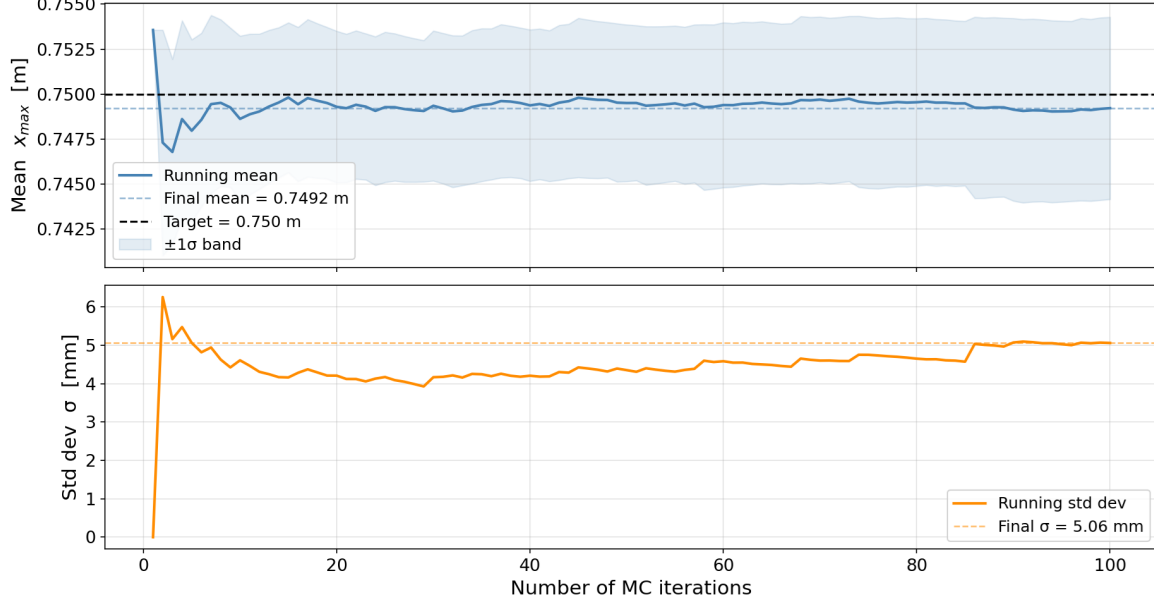


Figure 5: MC Mean and Std Convergence

derived control parameters still induce a robust, stable resonant behavior that effectively tolerates physical hardware uncertainties.

4 Model Generalization & Genetic Algorithm

A key aspect of this study was to develop a generalized algorithm capable of adapting to various robots with distinct physical parameters. To achieve this, an alternative approach combining a Forward Neural Network (NN) with a Genetic Algorithm (GA) was implemented. The underlying model, assumptions, and optimization strategy are detailed in the following sections.

4.1 Scaling of physical values

To train a universally applicable model, a robot-dependent dataset was generated. For each sample in the dataset, a unique robot was simulated with randomly assigned mass (M_r , M_b) and stiffness (K_r , K_b) parameters for both the robot and the base. The ranges are summarized in Table 5. For simplicity, the damping ratios were held constant at $\zeta_r = 0.3$ and $\zeta_b = 0.4$, respectively.

To isolate the fundamental physics of the system from the specific magnitude of the parameters, the chirp signal variables were non-dimensionalized. The dataset maps the input frequencies as a ratio between the initial chirp frequency f_0 and the true natural frequency of the 2-DOF system f_{nat} :

$$r = \frac{f_0}{f_{nat}}$$

Furthermore, rather than predicting absolute maximum displacement, the neural network was trained to predict the Dynamic Amplification Factor (DAF). Because the physical input parameter is a kinematic

Physical Parameters	
$M_b = 20 - 150 \text{ kg}$	$M_r = 5 - 65 \text{ kg}$
$K_b = 500 - 5000 \text{ N/m}$	$K_r = 100 - 3500 \text{ N/m}$
$\zeta_b = 0.4$	$\zeta_r = 0.3$

Table 5: Physical parameters range for robot and base

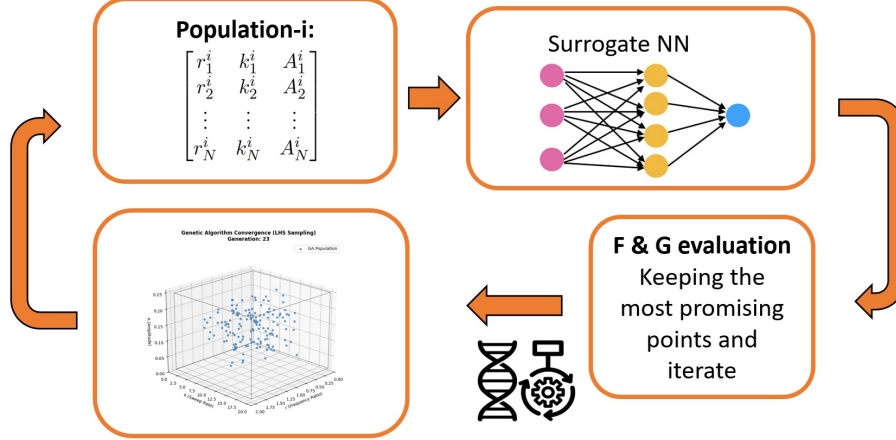


Figure 6: Optimization Algorithm with GA

displacement amplitude (A), the static deflection is simply equal to A . Therefore, the DAF is defined as:

$$DAF = \frac{x_{max} - x_{r0}}{A}$$

By structuring the inputs and outputs this way, the Neural Network learns the universal dynamic behavior of the system, independent of the real-world mass and stiffness values. The Neural Network was trained on a dataset of 20,000 episodes generated via numerical integration of the system's dynamics, utilizing mini-batches over 5,000 epochs. Both the Training and Validation RMSE converged to values < 0.19 . Because the network predicts the dimensionless DAF, this error margin is low enough (few millimeters), making it highly suitable for approximating the underlying physics.

4.2 Genetic Algorithm (GA) Optimization

With the Neural Network weights frozen, the inverse problem is solved using a Genetic Algorithm. The GA acts as the optimizer, searching the bounded 3D space for the optimal dimensionless parameters (r, k, A).

At any given generation i , the GA evaluates a population of $N = 300$ candidate solutions. The algorithm is represented in Figure 6. The algorithm is formulated with the objective to Minimize the physical stroke amplitude A and the following inequality constraint $|x_{final} - x_{target}| < 0.001 \text{ m}$.

The initial population is generated using Latin Hypercube Sampling (LHS) to ensure the candidate points are uniformly distributed across the search space, preventing the algorithm from converging prematurely on local minima. The search space itself is physically bounded: $r \in [0.05, 2.0]$, $k \in [1.01, 20.0]$, and $A \in [0.01 \text{ m}, 0.25 \text{ m}]$. These bounds ensure the algorithm explores highly dynamic resonant regions while avoiding high-frequency ranges where the system mass would simply remain immobile.

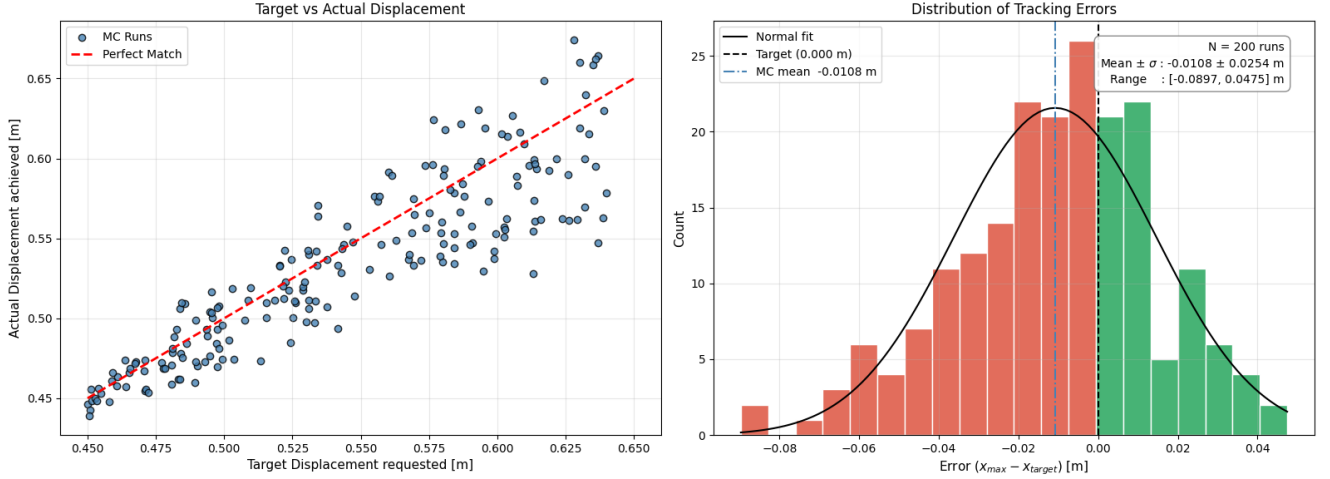


Figure 7: MC Analysis Results of the GA-model

4.3 Results and Verification

To rigorously assess the generalization of this implementation, a MC simulation was conducted over 200 iterations. In each iteration, the GA was tasked with predicting the optimal chirp parameters for a new robot configuration to hit a randomly generated x_{target} .

To ensure physical feasibility and prevent the algorithm from attempting to push the robot past its hard-stop boundaries, the target requests were strictly bounded between 0.45 m and 0.64 m. The results of the GA optimization were then verified against the true numerical dynamic simulator to compute the absolute error.

As presented in Figure 7, the mean physical error across all random configurations was around 1 cm. While this error is marginally larger than the one obtained using previous exact-numerical methods (due to the inherent approximation of the Neural Network), the computational time is drastically reduced. The time to run 50 generations of 300 sample each is less than 2 sec. By simulating the dynamics through a single forward pass of the NN to evaluate the population matrix instead of running hundreds of numerical integrations per generation, the GA can evaluate thousands of candidate solutions in a fraction of a second. This approach proves highly effective, allowing the algorithm to dynamically adapt to any robot configuration and instantly compute the optimal chirp parameters to reach a target while minimizing the required actuation amplitude.

5 Conclusions & Comments

To conclude, all approaches have demonstrated efficiency in finding solutions for the optimization of the chirp. The BO approaches showed an increased accuracy with the drawback of longer computational times. The two trained NNs demonstrated their capability to properly estimate the underlying physics of the system and drastically reduce computational times. Finally, the genetic algorithm demonstrated its capability to quickly compute the set of parameters while having a slightly higher error. In all cases, the minimization of the amplitude will lead to low force overshoots, keeping the robot safe in its environment.